QNX Interrupt & Exception Cheat Sheet to recall during BSP work.

QNX Interrupt & Exception Cheat Sheet (One-Page)

Embedded Systems • ARMv8-A • BSP • Firmware • RTOS

QNX Interrupt Model (Threaded Interrupt Architecture)

- **ISR = extremely short** (runs in interrupt context, no blocking, no message passing).
- **Handler thread = full processing** (runs in thread context, schedulable, preemptible).
- **InterruptAttach()** → custom ISR.
- **InterruptAttachEvent()** → generic ISR + event/pulse.
- **InterruptEnable/Disable** → enable/disable delivery.
- **InterruptMask/Unmask** → mask/unmask at controller level.
- **InterruptWait()** → thread blocks until interrupt event.
- **Pulse** → lightweight message used for ISR → thread signaling.
- **Shared interrupts** → multiple ISRs check device status.
- **ISR constraints** → no malloc, no sleep, no blocking, no IPC.

ARM GICv2 Integration (RPi4 / QNX BSP)

- **SGI** = Inter-processor interrupts (IPI).
- **PPI** = Per-CPU interrupts (timer, watchdog).
- **SPI** = External device interrupts.
- **GICC_IAR** → read interrupt ID.
- **GICC_EOIR** → end of interrupt.
- **GICC_PMR** → priority mask.
- **GICD_ITARGETSR** → route interrupts to CPUs.
- **Startup/BSP** configures GICD + GICC, priorities, routing, enable bits.

- **Timer interrupt** = PPI configured in BSP.
- **GPIO/PCIe/USB interrupts** = SPI configured in BSP.

QNX Real-Time Behavior

- **Determinism** via short ISRs + priority-based handler threads.
- **Preemption** allowed for higher-priority interrupts.
- **Nested interrupts** allowed based on GIC priority.
- **Interrupt storms** caused by unacknowledged device status.
- **Latency tuning** via handler thread priority + GIC priority.

QNX SMP Interrupt Handling

- **IPI (SGI)** used for TLB shootdown, scheduling.
- **Per-CPU PPIs** for timers.
- **Interrupt affinity** controlled via GIC routing.
- **Each core has its own interrupt context + vector entry.**

QNX Exception Model (ARMv8-A EL1 Kernel)

- QNX runs **entirely at EL1** (kernel) and **EL0** (user).
- No use of **EL2** (hypervisor) or **EL3** (secure monitor).
- All exceptions from EL0 → EL1.
- QNX always uses **SPx (SP_EL1)**, never SP0.

Exception Classes (ARMv8-A)

- **Synchronous** (page fault, alignment fault, illegal instruction, abort).
- **SVC** (system call).
- **IRQ** (interrupt).
- **FIQ** (unused in QNX).
- **SError** (fatal hardware error).

- **Debug** (breakpoint, watchpoint).

Exception Registers (Critical)

- **VBAR_EL1** → vector base.
- **SPSR_EL1** → saved PSTATE.
- **ELR_EL1** → return address.
- **ESR_EL1** → exception syndrome (EC + ISS).
- **FAR_EL1** → faulting address.
- **AFSR0/1_EL1** → auxiliary fault info.

Exception Vector Layout (EL1)

Offsets from **VBAR_EL1**:

- **0x200** → Sync (SPx)
- **0x280** → IRQ (SPx)
- **0x300** → FIQ (SPx)
- **0x380** → SError (SPx) (SP0 entries exist but unused by QNX.)

QNX Exception Handling Flow

EL0 → EL1 transition:

- CPU saves PSTATE → SPSR_EL1
- Saves PC → ELR_EL1
- Loads EL1 state
- Jumps to vector entry

Synchronous faults:

- Decode ESR_EL1
- Use FAR_EL1
- Send SIGSEGV/SIGBUS/SIGILL
- Kill process or panic kernel

SVC:

- ESR_EL1.ISS contains SVC number
- Kernel performs service
- ERET returns to EL0

IRQ:

- GIC → EL1 IRQ vector → QNX ISR → handler thread → EOI

SError:

- Fatal
- Log + panic + halt/reset

QNX BSP Exception Responsibilities

- Install vector table (VBAR_EL1).
- Configure MMU (TTBR0/1, TCR, SCTLR).
- Initialize GIC before enabling interrupts.
- Provide handlers for sync faults, IRQ, SError.
- Provide crash logs + ESR/FAR decoding.

High-Value Memory Items

- QNX = **threaded interrupt model**
- ISR = **minimal**, handler thread = **full work**
- GICC_IAR / GICC_EOIR = **core of IRQ handling**
- QNX always uses **SPx**, never SP0
- All exceptions handled at **EL1**
- ESR_EL1 = **root cause** of faults
- FAR_EL1 = **faulting address**
- SError = **fatal**
- SVC = **system call entry**
- QNX signals faults via **SIGSEGV/SIGBUS/SIGILL**